

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I S. Dharani Teja (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Perceptron – Failure to Converge

Problem: A perceptron is trained on linearly separable data but does not converge.

Possible causes:

- Learning rate may be too high or too low.
- Weights may be initialized incorrectly.
- Bias may be missing or incorrectly updated.
- The weight update rule may be implemented incorrectly.

Debugging:

1. Check the dataset to confirm it is linearly separable.
2. Print initial weights, bias, and learning rate.
3. Verify the prediction and error calculation.
4. Check whether weights are updated only when prediction is wrong.
5. Monitor the error after every epoch.

Optimization:

- Use a suitable learning rate.
- Normalize input features.
- Correctly initialize weights and bias.
- Stop training when there are no classification errors.

Result: These steps help identify implementation errors and provide faster and stable

convergence.

2. Multilayer Perceptron – Slow Convergence

Problem: An MLP using backpropagation converges slowly and gets stuck in local minima.

Possible causes:

- Vanishing gradients in deeper layers.
- Inappropriate activation functions.
- Poor weight initialization.
- Learning rate may be too high or too low.

Debugging:

1. Check training and validation loss after every epoch.
2. Monitor gradients in each layer.
3. Check the activation functions.
4. Test different weight initialization methods.
5. Experiment with different learning rates.

Optimization:

- Use ReLU or other suitable activation functions.
- Normalize the input data.
- Use adaptive learning rates.
- Apply optimizers such as Adam.
- Use proper weight initialization.

Result: These techniques improve gradient flow and allow the network to learn faster and more reliably.

3. Genetic Algorithm – Inconsistent Solutions

Problem: A genetic algorithm gives different and suboptimal solutions in different runs.

Possible causes:

- Incorrect chromosome encoding.
- Poorly designed fitness function.
- Premature convergence.
- Insufficient population diversity.

Debugging:

1. Verify the chromosome representation.
2. Test the fitness function with sample solutions.
3. Monitor population diversity in each generation.
4. Check selection, crossover, and mutation operations.
5. Compare results across multiple runs.

Optimization:

- Adjust the mutation rate to maintain diversity.
- Reduce excessive selection pressure.
- Use elitism to preserve the best solutions.
- Increase population size if necessary.

Result: Maintaining population diversity while preserving good solutions improves both solution quality and convergence speed.

4. Deep Neural Network + Genetic Programming – Overfitting

Problem: The combined model has high computational cost and performs poorly on unseen data.

Possible causes:

- The model is too complex.
- Too many parameters.
- Training data is insufficient.
- The model is overfitting the training data.

Debugging:

1. Compare training and validation accuracy.
2. Check the number of layers and parameters.
3. Identify unnecessary genetic-programming operations.
4. Check whether sufficient training data is available.

5. Measure training time and computational cost.

Optimization:

- Apply regularization to reduce overfitting.
- Use dropout.
- Prune unnecessary network connections or genetic-programming structures.
- Use efficient evaluation methods.
- Reduce unnecessary model complexity.

Result: Reducing complexity while using regularization and pruning can maintain accuracy while improving generalization and computational efficiency

5. Decision Tree – Poor Unseen-Data Performance

Problem: The decision tree has high training accuracy but poor test accuracy.

Possible causes:

- The tree is overfitting.
- The tree has excessive depth.
- The dataset contains noisy features or data.

Debugging:

1. Compare training and test accuracy.
2. Check the depth and number of nodes.
3. Identify branches based on noisy or irrelevant features.
4. Use cross-validation to evaluate generalization.
5. Analyze important features.

Optimization:

- Apply **pruning** to remove unnecessary branches.

- Use **cross-validation** to select suitable tree parameters.
- Perform **feature selection** to remove irrelevant features.
- Limit the maximum tree depth.

Result: A simpler tree with relevant features will reduce overfitting and improve performance on unseen data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand